
Cahier des Charges

Implémentation d'un Observatoire Centralisé des Services Numériques en DevOps

Préparé par :

LAOUINI Amen Allah

Stagiaire PFE

Filière : *informatique*

Encadré par :

M CHAARAOUI Chokri

Encadrant entreprise

Mme EL WARGHAMMI Radhia

Encadrant académique

Stage PFE — 6 mois

Janvier 2026 — Juillet 2026

Chapitre 1

Introduction Générale

1.1 Contexte du projet

Dans un contexte de transformation numérique accélérée, les entreprises déploient un nombre croissant de services numériques (applications web, APIs, microservices, bases de données, services cloud). La complexité de ces environnements rend la supervision manuelle **inadaptée et risquée**.

Constat

L'absence d'un système centralisé de monitoring et de gestion des logs engendre :

- Des temps de détection d'incidents élevés (MTTD ^a)
- Des diagnostics fragmentés entre plusieurs outils non corrélés
- Une incapacité à anticiper les montées en charge
- Un manque de visibilité pour le support technique et le management
- L'absence de mesures objectives de qualité de service (SLO/SLA)

a. Mean Time To Detect

1.2 Objectif du projet

Ce projet de stage PFE vise à **concevoir, implémenter et déployer un observatoire centralisé** des services numériques intégrant :

1. **La centralisation des logs** via la stack ELK (Elasticsearch, Logstash, Kibana)
2. **Le monitoring applicatif** via un outil d'APM (Application Performance Monitoring)
3. **Un système d'alerting intelligent** adapté aux périodes de montée en charge
4. **Des dashboards temps réel** pour le support technique
5. **Une chaîne de responsabilité** basée sur les Service Level Objectives (SLO)

1.3 Périmètre du stage

Élément	Description
Durée	6 mois (Janvier 2026 — Juillet 2026)
Étape 1	Benchmark des solutions existantes (mois 1–2)
Étape 2	Analyse et conception (mois 2–3)
Étape 3	Implémentation et déploiement (mois 3–5)
Étape 4	Tests, validation et documentation (mois 5–6)

Chapitre 2

Étape 1 : Benchmark des Solutions Existantes

2.1 Objectif du benchmark

L'objectif de cette étape est de réaliser une **étude comparative exhaustive** des solutions de monitoring, de centralisation de logs, d'APM et d'alerting disponibles sur le marché, afin de sélectionner la stack technologique la plus adaptée aux besoins de l'entreprise.

2.2 Domaines du benchmark

2.2.1 Centralisation des logs

Solution	Éditeur	Points forts	Points faibles
ELK Stack (Elasticsearch, Logstash, Kibana)	Elastic	Open source, très flexible, large communauté, scalable	Consommation mémoire élevée, complexité de configuration
Loki + Grafana	Grafana Labs	Léger, bon ratio coût/performance, intégration native Grafana	Moins mature que ELK, requêtes limitées comparé à Elasticsearch
Splunk	Splunk Inc.	Très puissant, fonctionnalités avancées, support entreprise	Coût de licence très élevé, propriétaire
Graylog	Graylog Inc.	Interface intuitive, open source (Community), alerting intégré	Moins scalable que ELK en très grands volumes
Fluentd / Fluent Bit	CNCF	Léger (Fluent Bit), plugins nombreux, cloud-native	Nécessite un backend (Elasticsearch, etc.)
Datadog Logs	Datadog	SaaS clé en main, corrélation logs/métriques/traces	Coût élevé à grande échelle, dépendance cloud

2.2.2 Monitoring applicatif (APM)

Solution	Éditeur	Points forts	Points faibles
Elastic APM	Elastic	Intégration native ELK, open source, distributed tracing	Configuration initiale complexe
Jaeger	CNCF / Uber	Open source, conçu pour le distributed tracing, cloud-native	Focalisé tracing uniquement, pas de métriques intégrées
Zipkin	Open source	Simple, léger, bon pour débuter	Fonctionnalités limitées vs Jaeger
Datadog APM	Datadog	SaaS complet, corrélation automatique, IA intégrée	Coût élevé, vendor lock-in
New Relic	New Relic	Interface riche, alerting avancé, full-stack observability	Modèle de pricing complexe
Dynatrace	Dynatrace	IA (Davis), auto-discovery, très entreprise	Très coûteux, courbe d'apprentissage
OpenTelemetry	CNCF	Standard ouvert, vendor-neutral, traces+métriques+logs	Framework (pas un produit fini), nécessite backend

2.2.3 Monitoring d'infrastructure et métriques

Solution	Éditeur	Points forts	Points faibles
Prometheus + Grafana	CNCF / Grafana Labs	Standard de facto Kubernetes, pull model, PromQL puissant	Stockage local limité, pas de clustering natif
Zabbix	Zabbix LLC	Mature, complet (réseau+serveur+app), templates riches	Interface vieillissante, configuration lourde
Nagios / Icinga	Open source	Très éprouvé, large communauté	Architecture datée, configuration fichiers plats
InfluxDB + Telegraf	InfluxData	Optimisé time-series, performant en écriture	Écosystème plus restreint
VictoriaMetrics	VictoriaMetrics	Compatible Prometheus, très performant, clustering	Communauté plus petite

2.2.4 Alerting

Solution	Éditeur	Points forts	Points faibles
Alertmanager (Prometheus)	CNCF	Intégration native Prometheus, routing avancé, silencing	Limité à Prometheus
ElastAlert 2	Open source	Intégration Elasticsearch, règles flexibles en YAML	Maintenance communautaire variable
Grafana Alerting	Grafana Labs	Multi-datasource, unifié dans Grafana, contact points multiples	Relativement récent, en évolution
PagerDuty	PagerDuty	On-call management, escalation policies, intégrations riches	Coût élevé, SaaS uniquement
OpsGenie	Atlassian	Intégration Jira/Confluence, on-call scheduling	Dépendance écosystème Atlassian

2.3 Critères d'évaluation du benchmark

Critère	Description	Poids (%)
C1 — Coût	Licence, infrastructure, maintenance	20
C2 — Open Source	Disponibilité du code source, communauté active	15
C3 — Scalabilité	Capacité à monter en charge (volume de données)	15
C4 — Intégrabilité	Facilité d'intégration avec l'écosystème existant	15
C5 — Facilité	Courbe d'apprentissage, documentation	10
C6 — Performance	Temps d'ingestion, latence des requêtes	10
C7 — Communauté	Taille de la communauté, fréquence des mises à jour	5
C8 — Sécurité	Chiffrement, RBAC, audit logs	10
Total		100

2.4 Grille de notation

Chaque critère est noté de **1 à 5** :

Note	Signification
1	Inadapté / Non supporté
2	Partiellement adapté / Support limité
3	Acceptable / Support standard
4	Bien adapté / Bon support
5	Excellent / Support complet

2.5 Matrice comparative

Résultats du benchmark

Les notes ci-dessous sont attribuées sur la base de l'analyse documentaire, des retours communautaires et des Proof of Concept réalisés. Chaque note est pondérée par le poids du critère pour obtenir le score total.

2.5.1 Centralisation des logs

Solution	C1	C2	C3	C4	C5	C6	C7	C8	Total
ELK Stack	4	3	5	5	3	5	5	4	4.20
Loki+Grafana	5	5	4	4	4	3	4	3	4.05
Splunk	1	1	5	4	3	5	4	5	3.10
Graylog	4	3	3	3	4	3	3	3	3.30
Datadog Logs	2	1	5	4	5	4	3	4	3.30

Pondération : C1 (20%), C2 (15%), C3 (15%), C4 (15%), C5 (10%), C6 (10%), C7 (5%), C8 (10%)

Résultat — Centralisation des logs

Solution retenue : ELK Stack (score 4.20/5) — Full-text search puissant, écosystème complet (APM, SIEM, ML intégrés), scalabilité éprouvée, large communauté.

Alternative : Loki + Grafana (4.05/5) en cas de contrainte forte de ressources serveur.

2.5.2 Monitoring applicatif (APM)

Solution	C1	C2	C3	C4	C5	C6	C7	C8	Total
Elastic APM	4	3	5	5	4	4	5	4	4.20
Jaeger	5	5	4	3	3	4	4	3	3.85
Zipkin	5	5	3	3	3	3	3	2	3.50
Datadog APM	2	1	5	4	5	5	3	4	3.45
New Relic	3	1	5	4	4	4	3	4	3.40
Dynatrace	1	1	5	4	5	5	3	5	3.30
OpenTelemetry*	5	5	5	5	3	4	5	3	4.40

* OpenTelemetry est un standard d'instrumentation (complémentaire), pas un produit concurrent direct.

Résultat — APM

Solution retenue : Elastic APM + OpenTelemetry — Intégration native ELK (zéro coût additionnel), corrélation automatique logs ↔ traces. OpenTelemetry comme couche d'instrumentation vendor-neutral.

Alternative : Jaeger (3.85/5) pour du tracing purement open source.

2.5.3 Monitoring d'infrastructure et métriques

Solution	C1	C2	C3	C4	C5	C6	C7	C8	Total
Prometheus + Grafana	5	5	4	5	4	5	5	3	4.55
Zabbix	5	5	3	3	3	3	4	4	3.75
Nagios / Icinga	5	5	2	2	2	3	3	3	3.25
InfluxDB + Telegraf	4	4	4	3	3	5	3	3	3.65
VictoriaMetrics	5	4	5	4	3	5	3	3	4.15

Résultat — Monitoring infrastructure

Solution retenue : Prometheus + Grafana (score 4.55/5) — Standard de facto Kubernetes, PromQL puissant, >200 exporters, 100% gratuit, dashboards Grafana de référence.

Complément : Metricbeat (Elastic) pour les métriques système centralisées dans Elasticsearch.

2.5.4 Alerting et gestion des incidents

Solution	C1	C2	C3	C4	C5	C6	C7	C8	Total
ElastAlert 2	5	4	3	5	3	4	3	3	3.90
Alertmanager	5	5	4	4	4	4	5	3	4.20
Grafana Alerting	5	4	4	4	4	3	4	3	3.95
PagerDuty	1	1	5	4	5	4	3	5	3.10
OpsGenie	2	1	4	4	4	4	3	4	3.15

Résultat — Alerting

Solutions retenues : ElastAlert 2 + Alertmanager (Prometheus) — ElastAlert 2 pour l'alerting sur les données Elasticsearch (logs + APM), Alertmanager pour les alertes sur les métriques Prometheus. Combinaison complémentaire couvrant tous les signaux.

Alternative : Grafana Alerting (3.95/5) comme solution unifiée si simplification souhaitée.

2.5.5 Synthèse globale — Stack technologique retenue

Domaine	Solution retenue	Score
Centralisation des logs	ELK Stack (Elasticsearch + Logstash + Kibana + Filebeat)	4.20/5
Monitoring applicatif	Elastic APM + OpenTelemetry	4.20/5
Monitoring infrastructure	Prometheus + Grafana + Metricbeat	4.55/5
Alerting	ElastAlert 2 + Alertmanager	4.05/5

Chapitre 3

Étape 2 : Analyse et Conception

3.1 Analyse de l'existant

3.1.1 Identification des problèmes actuels

1. **Fragmentation des logs** : logs dispersés sur plusieurs serveurs, pas de vue unifiée
2. **Détection tardive des incidents** : pas d'alerting proactif, détection souvent par les utilisateurs
3. **Absence de métriques APM** : pas de visibilité sur les performances applicatives
4. **Pas de SLO définis** : absence d'objectifs mesurables de qualité de service
5. **Support technique aveugle** : le support n'a pas de dashboards pour diagnostiquer

3.2 Analyse des besoins

3.2.1 Besoins fonctionnels

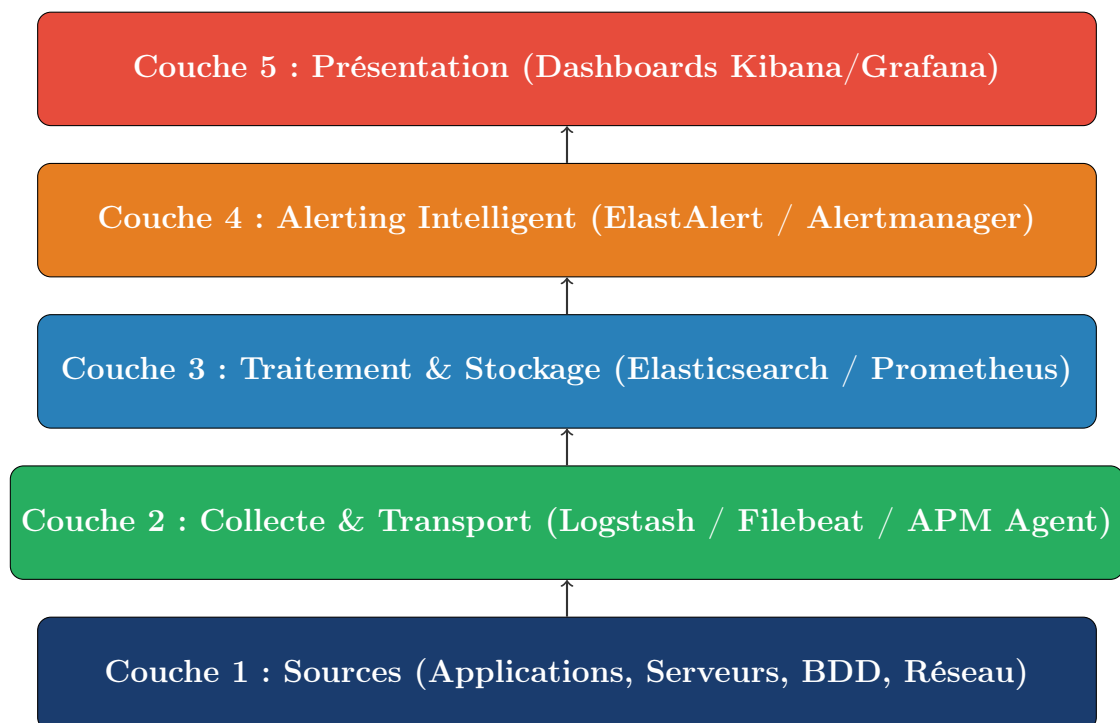
ID	Besoin fonctionnel	Priorité
BF-01	Centraliser les logs de toutes les applications et services dans un référentiel unique	Haute
BF-02	Permettre la recherche full-text et le filtrage avancé dans les logs	Haute
BF-03	Collecter les métriques APM (temps de réponse, taux d'erreur, throughput)	Haute
BF-04	Tracer les requêtes de bout en bout (distributed tracing)	Haute
BF-05	Configurer des alertes basées sur des seuils statiques et dynamiques	Haute
BF-06	Adapter l'alerting aux périodes de montée en charge (seuils contextuels)	Moyenne
BF-07	Fournir des dashboards temps réel pour le support technique (N1/N2/N3)	Haute
BF-08	Définir et suivre les SLO (disponibilité, latence, taux d'erreur)	Haute
BF-09	Générer des rapports périodiques (quotidien, hebdomadaire, mensuel)	Moyenne
BF-10	Permettre la corrélation entre logs, métriques et traces	Moyenne
BF-11	Gérer les escalades d'alertes selon la chaîne de responsabilité	Moyenne
BF-12	Supporter l'ajout dynamique de nouveaux services à monitorer	Basse

3.2.2 Besoins non fonctionnels

ID	Besoin non fonctionnel	Priorité
BNF-01	Performance : Ingestion de >10 000 logs/seconde sans dégradation	Haute
BNF-02	Disponibilité : L'observatoire doit avoir une disponibilité de 99.5%	Haute
BNF-03	Scalabilité : Architecture capable de supporter une croissance 3x du volume	Haute
BNF-04	Sécurité : Authentification, RBAC, chiffrement TLS, audit des accès	Haute
BNF-05	Rétention : Conservation des logs 30j chaud, 90j tiède, 1 an froid	Moyenne
BNF-06	Latence : Recherche dans les logs en <5 secondes pour les 7 derniers jours	Haute
BNF-07	Maintenabilité : Infrastructure as Code (IaC), documentation complète	Moyenne
BNF-08	Compatibilité : Support multi-OS (Linux), multi-langage (Java, Python, Node.js, Go)	Moyenne

3.3 Conception architecturale

3.3.1 Architecture globale de la solution



3.3.2 Architecture technique détaillée

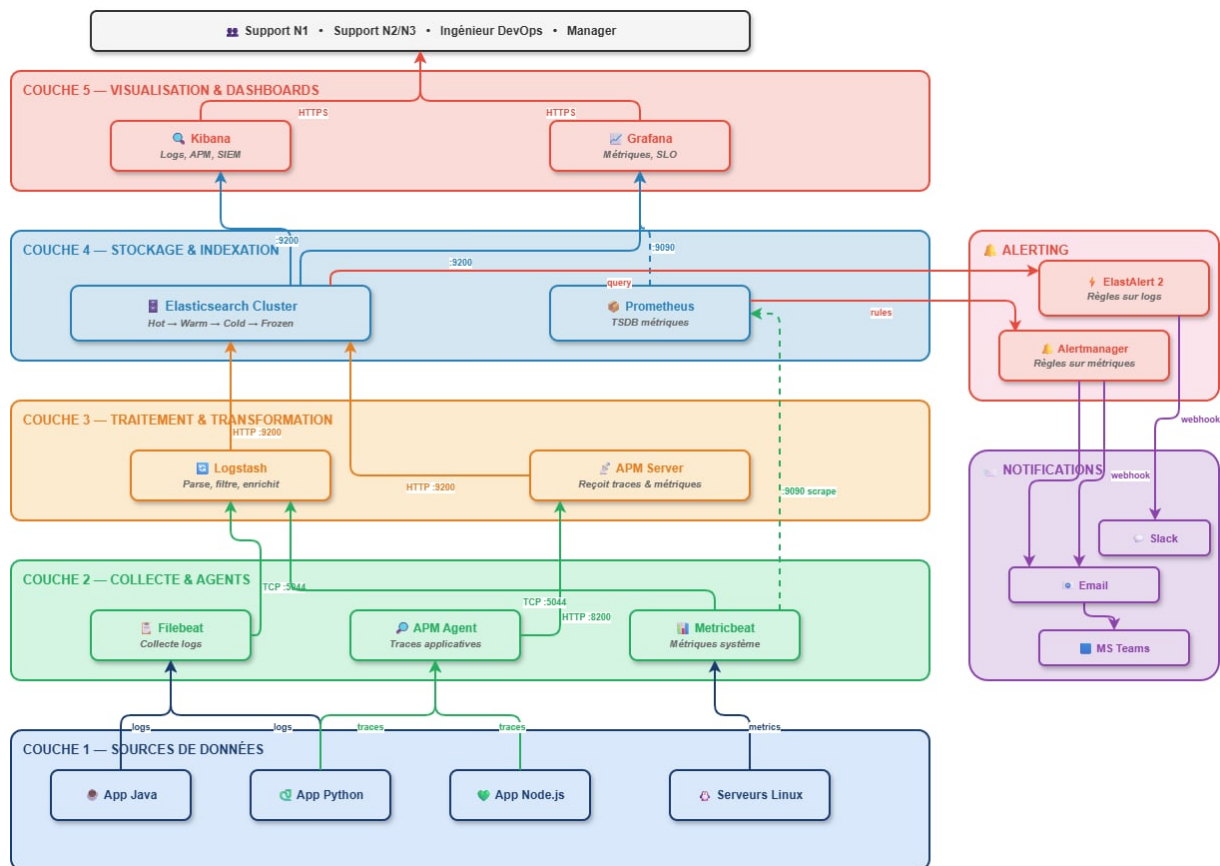


FIGURE 3.1 – Architecture technique détaillée de l'observatoire centralisé

3.3.3 Description des composants

Composant	Technologie	Rôle
Collecteur de logs	Filebeat / Fluent Bit	Agent léger installé sur chaque serveur, collecte et forward les logs
Agent APM	Elastic APM Agent	Intégré dans le code applicatif, collecte les traces et métriques
Collecteur métriques	Metricbeat	Collecte les métriques système (CPU, RAM, disque, réseau)
Pipeline	Logstash	Parse, filtre, enrichit et transforme les données avant stockage
Moteur de stockage	Elasticsearch	Indexation et recherche full-text des logs et métriques
Serveur APM	Elastic APM Server	Reçoit les données APM et les indexe dans Elasticsearch
Visualisation	Kibana + Grafana	Dashboards, visualisations, exploration des données
Alerting	ElastAlert 2	Règles d'alerting basées sur les données Elasticsearch
Notifications	Slack / Email / Teams	Canaux de notification pour les alertes

3.4 Conception de l'alerting intelligent

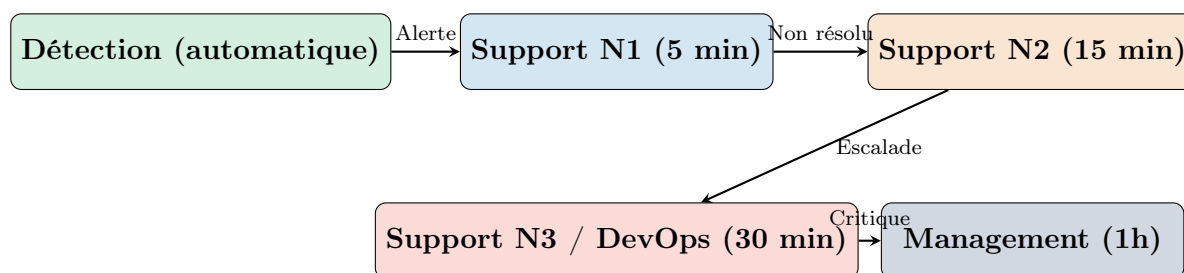
3.4.1 Stratégie d'alerting

Principe clé

L'alerting ne doit pas simplement reposer sur des seuils statiques. Il doit être **contextuel** et **adaptatif** en tenant compte des périodes de montée en charge.

Type d'alerte	Description	Exemple
Seuil statique	Alerte si une métrique dépasse un seuil fixe	CPU > 90% pendant 5 min
Seuil dynamique	Seuil basé sur la moyenne mobile ou l'écart-type	Temps de réponse > moyenne_7j + 2σ
Anomalie	Détection de patterns anormaux via ML	Pic de logs d'erreur 500 inhabituel
Calendaire	Seuils différents selon le créneau horaire	Seuil relâché pendant les batchs nocturnes
Composite	Alerte combinant plusieurs conditions	CPU > 80% ET mémoire > 85% ET latence > 2s

3.4.2 Chaîne d'escalade



3.5 Conception des SLO (Service Level Objectives)

Service	SLI	SLO	Mesure
API principale	Disponibilité	99.9%	Ratio requêtes réussies / total
API principale	Latence P99	< 500ms	99e percentile temps de réponse
Application web	Taux d'erreur	< 0.1%	Ratio erreurs 5xx / total requêtes
Base de données	Temps P95	< 100ms	95e percentile des requêtes SQL
Service batch	Complétion	100%	Tous les jobs terminés sans erreur

Error Budget

Chaque SLO est associé à un **error budget**. Par exemple, un SLO de 99.9% de disponibilité sur 30 jours autorise **43.2 minutes** d'indisponibilité. Les dashboards afficheront le pourcentage d'error budget consommé en temps réel.

3.6 Conception des dashboards

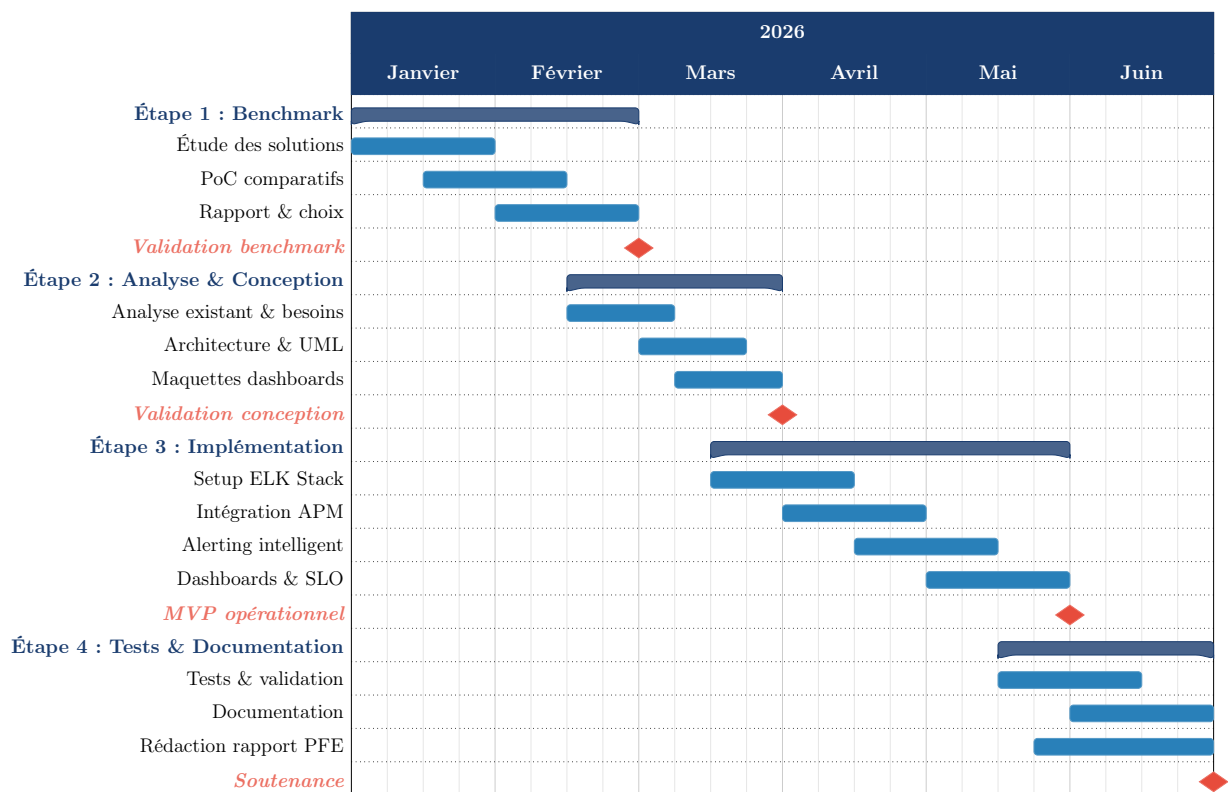
3.6.1 Dashboard — Vue globale

Widget	Description
Health Map	Carte de santé de tous les services (vert/jaune/rouge)
Alertes actives	Liste des alertes en cours avec sévérité et durée
Top erreurs	Top 10 des erreurs les plus fréquentes (dernière heure)
Trafic temps réel	Graphe du nombre de requêtes par seconde
SLO Status	Barre de progression de chaque SLO avec error budget restant

Chapitre 4

Planification Globale du Projet

4.1 Planning prévisionnel



Chapitre 5

Annexes

5.1 Références bibliographiques

1. Elastic, *Elasticsearch Reference*, <https://www.elastic.co/guide/en/elasticsearch/reference/current/>
2. Elastic, *APM Guide*, <https://www.elastic.co/guide/en/apm/guide/current/>
3. Google SRE, *Site Reliability Engineering*, <https://sre.google/sre-book/table-of-contents/>
4. Google SRE, *The Site Reliability Workbook*, <https://sre.google/workbook/table-of-contents>
5. Grafana Labs, *Grafana Documentation*, <https://grafana.com/docs/>
6. CNCF, *OpenTelemetry Documentation*, <https://opentelemetry.io/docs/>
7. Prometheus, *Prometheus Documentation*, <https://prometheus.io/docs/>